
AXIOM

Retargetable Compiler Backend

Legendary Optimization Blueprint

A comprehensive deep-dive analysis of the Axiom codebase identifying every gap between its innovative architecture and production-grade compiler quality, with a prioritized roadmap to transform it into the most optimized backend in compiler history.

Z.ai | June 2026

1. Executive Summary

Axiom is a retargetable compiler backend written in Rust that employs a sea-of-nodes intermediate representation with first-class OwnershipRoot semantics for guaranteed no-aliasing. It targets x86_64, AArch64, RISC-V, and WebAssembly, and includes an autotuner, JIT/AOT hybrid compilation, ThinLTO, and profile-guided block layout. The architectural vision is genuinely innovative: by making no-aliasing a structural property rather than an analysis result, Axiom eliminates the single biggest optimization barrier in LLVM. The theoretical performance ceiling for memory-heavy code is 2-5x faster than LLVM, with 20-40% more register availability through ownership-aware register allocation.

However, after an exhaustive line-by-line audit of all 21 crates and every source file in the repository, this blueprint identifies a significant gap between the architectural vision and the current implementation maturity. Several critical systems exist only as scaffolding: the x86_64 vector instruction selection emits placeholders, the register allocator lacks the VRegScanner integration needed for proper live intervals, the MIR lowering collapses all control flow into a single basic block, and the loop vectorizer is a stub that always returns false. These gaps mean that while Axiom's design is architecturally sound, it cannot yet produce competitive machine code for real workloads.

This document presents a prioritized, phased roadmap to close these gaps and then push beyond them into territory that no existing compiler backend has explored. The plan is organized into four phases: Critical Path (making the backend functional), Performance Foundation (making it competitive), Advanced Capabilities (making it distinctive), and Legendary Status (making it historically significant). Each item includes specific file paths, code-level recommendations, and estimated impact.

Phase	Timeline	Focus	Impact
Phase 1: Critical Path	Months 1-3	Functional codegen for all targets	Can compile real programs
Phase 2: Performance Foundation	Months 4-6	Competitive optimization quality	Within 10-15% of LLVM -O2
Phase 3: Advanced Capabilities	Months 7-12	Distinctive ownership-driven features	Beats LLVM on memory-heavy workloads
Phase 4: Legendary Status	Months 12-18	Research-grade innovations	2-5x on targeted workloads, new capabilities

2. Current State Analysis

2.1 What Axiom Gets Right

The OwnershipRoot system is Axiom's crown jewel and represents a genuine paradigm shift. In LLVM, proving that two memory operations don't alias requires TBAA, SCEV, restrict qualifiers, and interprocedural analysis that is imprecise and often fails. In Axiom, different roots simply cannot alias by construction. This makes CSE, DSE, memory scheduling, register allocation, and cross-module inlining trivially correct without alias analysis. The ownership-aware DSE pass in `crates/axiom-opt/src/dse.rs` is

a perfect demonstration: checking whether a store is dead requires only comparing root IDs, not traversing alias chains. This is simpler, faster, and more precise than anything LLVM can do.

The sea-of-nodes IR design directly enables the ownership system. Because each data node is an independent entity with explicit dependencies, per-root memory chains are a natural fit. The IR node types in `crates/axiom-ir/src/nodes.rs` include first-class vector operations (`VecLoad`, `VecStore`, `VecGather`, `VecScatter`, `VecReduce`, `VecShuffle`) that carry `OwnershipRoot`, meaning the no-aliasing guarantee extends seamlessly to SIMD code. This is a feature no other compiler provides.

The autotuner in `crates/axiom-autotuner` is architecturally sophisticated, with beam search, stochastic exploration, an ownership-aware cost model, and schedule search. The ThinLTO implementation makes ownership-aware import decisions, which is a novel optimization. The JIT tiered compilation model with deoptimization mirrors HotSpot C2 and GraalVM, and the interpreter in `crates/axiom-jit/src/interpreter.rs` can actually execute IR graphs. The profile-guided block layout implements the Pettis-Hansen algorithm correctly.

2.2 Critical Gaps (Must Fix for Production)

Gap 1: Single-Block MIR Lowering

The MIR lowering in `crates/axiom-mir/src/lower.rs` puts ALL instructions into blocks[0]. There is no multi-block CFG construction from the sea-of-nodes IR. The control nodes (`If`, `Jump`, `Return`) exist in the IR but are never translated into separate MIR basic blocks. Phi nodes are lowered by emitting all moves sequentially in block 0, which is incorrect for any function with branches or loops. This means Axiom cannot compile any program with control flow beyond straight-line code, which is every real program. The impact is catastrophic: the entire backend produces incorrect code for anything beyond trivial functions.

[CRITICAL] MIR lowering collapses all control flow into one block. Branches, loops, and conditionals do not work.

Gap 2: x86_64 Vector ISel Is Placeholder

The `x86_64` instruction selection in `crates/axiom-x86/src/isel.rs` has vector operations as stubs. `VecBroadcast` emits `MovR1` with immediate 0 (line 142). `VecLoad` and `VecStore` emit scalar `Load/Store` instructions (lines 148-165). `VecReduce`, `VecShuffle`, `VecGather`, and `VecScatter` are complete no-ops (lines 166-182). `ExtractLane` and `InsertLane` have empty bodies (lines 129-136). Since first-class vector IR is Axiom's flagship differentiator, the fact that the backend cannot actually emit any vector instructions means this entire value proposition is unrealized.

[CRITICAL] x86_64 vector ISel emits scalar stubs. No SSE/AVX instructions are generated. The SIMD story is fictional at the codegen level.

Gap 3: X86Inst Does Not Implement VRegScanner

The register allocator in `crates/axiom-regalloc/src/lib.rs` defines a `VRegScanner` trait (line 133) that target instruction types should implement to enable proper live interval construction. `X86Inst` in `crates/axiom-x86/src/inst.rs` does NOT implement this trait. As a result, the codegen driver in `crates/axiom-codegen/src/lib.rs` uses `allocate_func_conservative` (line 58), which makes every virtual register's live interval span the entire function. This destroys register efficiency: instead of freeing registers as soon as their last use ends, all registers are held for the entire function duration, causing unnecessary spills and wasting 50-80% of available registers on typical code.

[CRITICAL] Without VRegScanner, all vreg live intervals span the entire function. This causes 50-80% register waste and excessive spilling.

Gap 4: Loop Vectorizer Is a Stub

The loop vectorizer in `crates/axiom-opt/src/loop_vectorize.rs` is a 36-line file that contains only documentation and a function that returns false. While the philosophy of 'autovectorization is a bonus, not a promise' is sound, having zero loop vectorization means Axiom cannot automatically optimize the most common parallelizable pattern in existence (for i in $0..n$ { $a[i] = b[i] + c[i]$ }). The SLP vectorizer handles straight-line code but cannot handle loops.

[HIGH] Loop vectorizer is a stub. The most common vectorization pattern (map loops) is completely unhandled.

Gap 5: Missing IR Operations

The IR BinOp enum in `crates/axiom-ir/src/operators.rs` has only 13 variants (Add, Sub, Mul, Div, Mod, Eq, Ne, Lt, Gt, Le, Ge, And, Or). It is missing Shl, Shr, Sra (arithmetic shift right), and Xor. These operations appear in the MIR BinOp (`crates/axiom-target/src/mir_types/types.rs`) but cannot be expressed in the IR. Additionally, there are no floating-point operations at all. The LaneType enum supports F32 and F64, but there are no FP constants, no FP BinOp variants (FAdd, FSub, FMul, FDiv), and no FP comparison semantics. This means Axiom cannot compile any floating-point code, which rules out scientific computing, games, ML inference, and most real workloads.

[HIGH] Missing Shl/Shr/Sra/Xor in IR. No floating-point support. Cannot compile most real programs.

Gap 6: Shift Amounts Hardcoded to 1

In the `x86_64` instruction selection (`crates/axiom-x86/src/isel.rs`, lines 269-282), the ShlRI, ShrRI, and SarRI instructions are emitted with a hardcoded amount of 1. This means $x = x \ll 3$ would compile as $x = x \ll 1$, producing completely wrong code. The MIR has no mechanism to carry the shift amount because the IR has no shift operations (Gap 5), so even if VReg-based shift amounts were available in MIR, they cannot reach the ISel.

[HIGH] Shift amounts hardcoded to 1 in x86 ISel. Any code using shifts produces wrong results.

2.3 Performance Gaps

Gap 7: No Instruction Scheduling from Sea-of-Nodes

The sea-of-nodes IR is fundamentally unordered: nodes exist without a predefined execution sequence, and a scheduler must determine the optimal instruction order. Currently, Axiom uses the insertion order of `data_nodes` as the de facto schedule, which is whatever order the `IrBuilder` happened to create them. This defeats the entire purpose of sea-of-nodes: without scheduling, the compiler cannot exploit instruction-level parallelism, minimize register pressure, or respect latency constraints. The autotuner has a `StochasticScheduler` (`crates/axiom-autotuner/src/schedule.rs`), but it is not integrated into the main compilation pipeline.

Gap 8: No Peephole Optimizations

The `x86_64` emitter (`crates/axiom-x86/src/emit.rs`) translates each `X86Inst` directly to assembly without any peephole pass. This means: `xor rax, rax` is not recognized as a zero idiom (though the `xor reg, reg` pattern is present, it is not optimized). `MovRR` from a register to itself is not eliminated. `LEA` is not used

for arithmetic (add rax, 5 when the result could use lea rax, [rbx+5]). cmp + jcc macro-op fusion opportunities are not exploited. These are table-stakes optimizations that every production compiler performs.

Gap 9: Strength Reduction Is Minimal

The strength reduction pass in crates/axiom-opt/src/strength_reduce.rs only handles modulo by power-of-2 ($x \% N$ to $x \& (N-1)$). It does not handle: multiplication by constant ($x * 15$ to $x \ll 4 - x$), division by constant (the magic multiply transform that the README mentions but doesn't implement), or addition/subtraction by constant with LEA opportunity. On x86 where division is 25-35 cycles but multiplication is 3 cycles, the magic multiply transform for constant divisors is one of the most impactful single optimizations available.

Gap 10: CSE Uses String-Based Signatures

The CSE pass in crates/axiom-opt/src/cse.rs computes node signatures by formatting to strings (line 127: `format!("{}", op, left, right)`). This is $O(n)$ per node per comparison where n is the string length, and it allocates a heap string for every node. A production CSE uses hash-based signatures (e.g., combining op code and operand IDs into a u64 hash) for $O(1)$ comparison. For large functions with thousands of nodes, the string-based approach causes significant compile-time overhead.

3. Phase 1: Critical Path (Months 1-3)

Goal: Make Axiom capable of compiling real programs with correct control flow and working vector codegen on x86_64.

3.1 Multi-Block MIR Lowering

This is the single highest-priority item. Without multi-block CFG construction, Axiom is fundamentally broken for any non-trivial program. The fix requires: (a) Walking the control nodes in the IrGraph to identify basic block boundaries. (b) Creating a MirBlock for each distinct control flow path. (c) Lowering If nodes to Brlf terminators that branch to the correct MIR blocks. (d) Lowering Jump nodes to Jump terminators. (e) Placing Phi-resolution moves at the end of predecessor blocks rather than sequentially in block 0. (f) Building proper successor/predecessor lists for each block. The current lower_function in crates/axiom-mir/src/lower.rs should be refactored to first build a CFG from the control nodes, then lower data nodes into their respective blocks.

3.2 Complete x86_64 Vector ISel

The vector ISel must be upgraded from stubs to real instruction selection. Priority order: (a) VecBroadcast to vbroadcastss/vbroadcastsd (AVX2) or movddup (SSE3). (b) VecLoad to vmovaps/vmovupd (aligned/unaligned). (c) VecStore to vmovaps/vmovupd. (d) VecBinOp to vpaddd/vpsubd/vpmulld etc. based on op and lane size. (e) ExtractLane to pextrd/pextrq. (f) InsertLane to pinsrd/pinsrq. (g) VecReduce to horizontal add sequences (phadd or shufps + addps pattern). (h) VecShuffle to pshufd/shufps. This requires adding XMM/YMM register class support to the register allocator and target description. The X86XMM register enum already exists in

crates/axiom-x86/src/registers.rs but is not connected to the regalloc.

3.3 Implement VRegScanner for X86Inst

X86Inst in crates/axiom-x86/src/inst.rs must implement the VRegScanner trait. Each instruction variant knows which virtual registers it uses and defines. For example: MovRR uses src and defines dst. AddRR uses src and dst (2-op form), defines dst. IMulRR uses lhs and rhs, defines dst. CmpRR uses lhs and rhs, defines nothing (sets flags). Ret uses no vregs. Call uses arg vregs, defines return vreg. This is a straightforward pattern-match implementation on the X86Inst enum. Once implemented, the codegen driver should switch from `allocate_func_conservative` to `allocate_func` for proper live interval construction.

3.4 Add Missing IR Operations

Extend the BinOp enum in crates/axiom-ir/src/operators.rs to include Shl, Shr, Sra, and Xor. Add UnOp::Abs. Then add floating-point support: FP BinOp variants (FAdd, FSub, FMul, FDiv), FP comparison variants (FOEq, FOLt, FOGt, FOLe, FOGe, FONE), and an FP constant node (FloatConst(f64)). Update the IrBuilder, the ownership analysis, the MIR mapping, and all four target ISels. The MIR BinOp already has Shl, Shr, Sra, and Xor, so the mapping is straightforward. For FP, the MIR needs corresponding additions.

3.5 Fix Shift Amount Propagation

The shift amount must flow from the IR through MIR to ISel. Currently the IR has no shift ops (fixed in 3.4). The MIR BinOp for Shl/Shr/Sra should carry the amount as a second operand (rhs vreg), and the ISel should emit ShlRI/ShrRI/SarRI with the correct amount extracted from the rhs vreg's definition (if it is a MovImm). For variable shifts, the ISel should emit ShlRR/ShrRR/SarRR using the CL register per x86 convention.

4. Phase 2: Performance Foundation (Months 4-6)

Goal: Bring optimization quality to within 10-15% of LLVM -O2 on standard benchmarks.

4.1 Instruction Scheduler Integration

The sea-of-nodes IR requires an explicit scheduling pass before or during MIR lowering. The scheduler should: (a) Build a dependency DAG from the IR graph data nodes. (b) Use the OwnershipInfo to identify independent operations on different roots (these can be scheduled in parallel). (c) Apply a list scheduling algorithm that minimizes register pressure while respecting latency. (d) Use the StochasticScheduler from the autotuner for hot functions. The schedule determines the instruction order in the MIR, so it must happen before or during `lower_function`. The autotuner's ScheduleDAG can be adapted for this purpose.

4.2 Peephole Optimizer for x86_64

Add a target-specific peephole pass that runs after register allocation and before assembly emission. Key patterns: (a) Zero idiom: `xor reg, reg` replaces `mov reg, 0`. (b) Self-move elimination: `mov reg, reg` is a no-op. (c) LEA for arithmetic: `add reg, imm` becomes `lea reg, [reg + imm]` when it frees the flags register. (d) `cmp + jcc` macro-op fusion: ensure `cmp` immediately precedes `jcc`. (e) Test idiom: `test reg, reg` replaces `cmp reg, 0`. (f) Redundant `mov` elimination after `regalloc`. This pass should be implemented as a post-`regalloc` walk over `RegAllocBlock` instructions.

4.3 Complete Strength Reduction

Expand `crates/axiom-opt/src/strength_reduce.rs` with: (a) Multiplication by constant: $x * C$ where C is not a power of 2. Decompose into shift-and-add sequence (e.g., $x * 15 = (x \ll 4) - x$). (b) Division by constant: implement the magic multiply transform for unsigned and signed division. This is critical for x86 where `idiv` takes 25-35 cycles vs 3 for `imul`. (c) Modulo by constant: combine magic multiply division with multiplication and subtraction. (d) Power-of-2 multiplication: $x * N$ where N is a power of 2 becomes $x \ll \log_2(N)$. (e) Addition with LEA opportunity: annotate for the peephole pass.

4.4 Loop Vectorizer (Basic)

Implement a basic loop vectorizer in `crates/axiom-opt/src/loop_vectorize.rs` that handles the most common pattern: `for i in 0..n { a[i] = f(b[i], c[i]) }`. Requirements: (a) Detect loop structures via back-edge analysis through `Phi` and `Region` nodes. (b) Verify no loop-carried dependencies (except simple reductions). (c) Check for affine access patterns (`base + i * stride`). (d) Verify no control flow in the loop body. (e) Widen scalar operations to vector operations. (f) Generate the appropriate `VecLoad`, `VecBinOp`, `VecStore` sequence. (g) Handle remainder loops for non-aligned trip counts. Start with the map pattern only; reductions can follow.

4.5 Hash-Based CSE Signatures

Replace the string-based CSE signatures in `crates/axiom-opt/src/cse.rs` with u64 hashes. For example: `binop_signature = (op as u64) << 32 | (left.0 as u64) << 16 | (right.0 as u64)`. This eliminates all string allocation and reduces comparison from $O(n)$ to $O(1)$. The `HashMap` key type changes from `String` to `u64`. This is a compile-time optimization that matters for large functions.

4.6 AArch64 and RISC-V Backend Completion

The AArch64 and RISC-V backends currently have `ISel`, `emit`, and register descriptions but need the same level of completion as `x86_64` (after Phase 1 fixes). Specifically: (a) AArch64 needs proper NEON vector `ISel` (the AArch64 target already has `has_predication = true` and `has_3op_arithmetic = true` in its `TargetDesc`). (b) RISC-V needs V-extension vector `ISel`. (c) Both need `VRegScanner` implementations. (d) Both need calling convention handling in the `ISel` (argument passing via `a0-a7 / x10-x17`). (e) Both need proper prologue/epilogue generation with callee-save register handling.

5. Phase 3: Advanced Capabilities (Months 7-12)

Goal: Beat LLVM on memory-heavy workloads. Make the ownership-driven features deliver measurable performance advantages.

5.1 Ownership-Aware Global Instruction Scheduling

This is where Axiom's ownership system delivers its most powerful advantage. Implement a scheduler that: (a) Groups memory operations by OwnershipRoot. (b) Schedules operations on different roots in parallel (issue them to independent execution ports). (c) Reorders operations across roots freely (guaranteed safe by no-aliasing). (d) Minimizes memory dependency chain length within each root. (e) Prefetches data for roots that will be accessed soon. This is IMPOSSIBLE in LLVM because it cannot prove that operations on different allocations don't alias. In Axiom, this is trivially correct by construction. For memory-heavy workloads (databases, graph processing, data analytics), this alone can deliver 2-3x speedup over LLVM.

5.2 Ownership-Aware Register Allocation with Graph Coloring

Upgrade from linear scan to an ownership-aware graph coloring allocator. The key insight: values from different ownership roots have non-overlapping live ranges when viewed through the lens of memory dependency chains. This means: (a) Move points truncate live intervals (already implemented in LiveInterval::move_point). (b) Values that are moved can donate their registers immediately. (c) Memory chain values from different roots never interfere. (d) The interference graph is naturally sparser than LLVM's because ownership eliminates alias-based conservatism. A graph coloring allocator exploiting these properties should achieve 20-40% fewer spills than LLVM on the same code.

5.3 LICM (Loop-Invariant Code Motion) with Ownership

Implement loop-invariant code motion that leverages ownership information. If a load from root A is loop-invariant (the pointer and root don't change across iterations), it can be hoisted out of the loop. In LLVM, hoisting loads requires proving that no store in the loop could write to the loaded location (alias analysis). In Axiom, you only need to check that no store to root A exists in the loop. This is a simple root-ID comparison, not an alias analysis. The implementation should: (a) Identify loop bodies via back-edge analysis. (b) For each Load in the loop, check if any Store to the same root exists in the loop. (c) If not, hoist the Load before the loop. (d) Similarly for Stores: if a Store to root A writes the same value in every iteration, it can be sunk after the loop.

5.4 Escape Analysis and Stack Allocation

Implement escape analysis that determines whether an OwnershipRoot's data escapes the current function. If a root's data never escapes (no references stored to heap, passed to external functions, or returned), it can be stack-allocated instead of heap-allocated. The ownership system makes this analysis straightforward: trace the root's uses and check if any escape the function. This eliminates allocation overhead, improves cache locality, and enables more aggressive optimization of the stack-allocated data.

5.5 Tail Call Optimization

Implement tail call optimization for recursive functions in tail position. When a function's return value is the direct result of a call to another function (with no computation after the call), replace the call with a jump, reusing the current stack frame. This is critical for functional programming patterns and eliminates stack overflow for tail-recursive functions. The implementation should detect tail-position calls during

optimization and replace them with a TailCall IR node that lowers to a jump during ISel.

5.6 Speculative Devirtualization in JIT Tier

In the JIT tier (`CompilationTier::JitProfiled`), use profile data to speculatively devirtualize call sites. If a virtual call site always dispatches to the same implementation (monomorphic), replace the indirect call with a direct call to the hot implementation, guarded by a type check with a deoptimization point. This is one of the biggest advantages of JIT over AOT compilation, and Axiom's deoptimization infrastructure (`crates/axiom-jit/src/deopt.rs`) is already designed to support it.

6. Phase 4: Legendary Status (Months 12-18)

Goal: Push into territory no existing compiler has explored. Make Axiom the reference implementation for ownership-driven compilation.

6.1 Direct Machine Code Emission

Replace the `.s` text + `llvm-mc` assembly pipeline with direct binary encoding. For `x86_64`, implement an encoder that produces ELF64 object files directly. This eliminates the assembler dependency, reduces compile time by 15-30% (no process spawn, no text formatting/parsing), and enables relaxation and fixup handling in a single pass. The encoder should handle: (a) REX prefix generation. (b) ModR/M and SIB byte encoding. (c) Immediate encoding for all instruction forms. (d) Relocation records for external symbols. (e) ELF section and symbol table generation. Cranelift's encoder design is a good reference.

6.2 Parallel Function Compilation

Implement parallel compilation of independent functions using rayon or a work-stealing thread pool. The `IrCrate` is naturally a collection of independent functions. The optimization pipeline (ownership analysis, `optimize_graph`, `lower_graph`) is pure and thread-safe. The codegen pipeline (`legalize`, `isel`, `regalloc`, `emit`) is also per-function and independent. The only synchronization point is the ThinLTO import decision phase. This should reduce wall-clock compile time by 4-8x on 8-core machines for large codebases.

6.3 Incremental Compilation

Implement a caching system that avoids recompiling unchanged functions. Each function's IR graph can be hashed (including its ownership analysis and optimization results). If the hash matches a cached compilation, reuse the previous assembly output. This requires: (a) A stable hashing scheme for `IrGraph`. (b) A persistent on-disk cache (similar to Rust's incremental compilation artifact system). (c) Dependency tracking: if a function is inlined into another, changes to the inlined function invalidate the caller's cache. (d) Integration with the ThinLTO system for cross-module invalidation.

6.4 Ownership-Aware Auto-Vectorization with Masking

Implement advanced auto-vectorization that uses `OwnershipRoot` to enable masked vectorization of loops with conditionals. In LLVM, if-conversion is required before vectorization because alias analysis

cannot prove that stores in different branches don't conflict. In Axiom, operations on different roots never conflict, so masked vectorization is always safe. The implementation should: (a) Vectorize loops with if/else branches using predicated vector operations. (b) Generate mask vectors from condition values. (c) Use target predication support (AVX-512 opmask, ARM SVE, RISC-V V extension). (d) Handle gather/scatter for non-contiguous access patterns. (e) Exploit ownership to skip alias checks entirely.

6.5 Profile-Guided Type Specialization

In the JIT tier, implement type specialization based on runtime profiles. If a function parameter is observed to always have a specific type layout (e.g., a struct with two i64 fields), create a specialized version of the function that assumes this layout. This enables: (a) Inlining of method calls based on observed types. (b) Elimination of dynamic dispatch. (c) Field access optimization (known offsets, no indirection). (d) Unboxing of boxed values. This is the technique that makes V8's TurboFan and the JVM's C2 compiler produce code that often matches or beats C++ for hot paths.

6.6 DWARF Debug Info Generation

Implement DWARF 5 debug information generation so that programs compiled with Axiom can be debugged with standard tools (gdb, lldb, perf). This requires: (a) Line number mapping from IR nodes to source locations (extend the Span/DefId system). (b) Variable location lists (mapping from PC ranges to register/stack locations). (c) Type information for variables. (d) Function frame information (CFA, saved registers). (e) Integration with the ELF object writer for `.debug_*` sections. Without debug info, Axiom cannot be used for production development regardless of performance quality.

6.7 Ownership-Driven Prefetching

Implement a prefetching pass that inserts prefetch instructions for ownership roots that will be accessed in the near future. The ownership system provides precise information about which allocations will be accessed and in what order, making prefetch insertion far more accurate than hardware prefetchers. The implementation should: (a) Analyze the memory access pattern for each root in the scheduled instruction stream. (b) Insert prefetch instructions (PREFETCHT0/PREFETCHT1 on x86, PRFM on AArch64) ahead of the first use of each root. (c) Use profile data to determine optimal prefetch distance. (d) Avoid prefetching for roots that are already in cache (based on access frequency). This is a novel optimization that only Axiom can do because only Axiom has ownership-root semantics.

7. Implementation Priority Matrix

The following table ranks all identified work items by a composite score considering impact on generated code quality, correctness requirement, implementation effort, and dependency chain. Items with correctness impact (wrong code) rank highest regardless of effort.

Rank	Item	Phase	Impact	Effort	Dependencies
1	Multi-block MIR lowering	1	Correctness	M	None

Rank	Item	Phase	Impact	Effort	Dependencies
2	Missing IR ops (Shl/Shr/Sra/Xor/FP)	1	Correctness	M	None
3	Fix shift amount propagation	1	Correctness	S	#2
4	VRegScanner for X86Inst	1	Perf (50-80% reg waste)	M	None
5	x86_64 vector ISel	1	Perf (SIMD = 4-8x)	L	#4
6	Instruction scheduler	2	Perf (ILP)	L	#1
7	Peephole optimizer	2	Perf (5-10%)	M	None
8	Complete strength reduction	2	Perf (div = 10x)	M	None
9	Loop vectorizer (basic)	2	Perf (2-4x on loops)	L	#5
10	Hash-based CSE signatures	2	Compile time	S	None
11	AArch64/RISC-V completion	2	Target coverage	L	#1, #4
12	Ownership-aware scheduling	3	Perf (2-3x mem)	L	#6
13	Graph coloring regalloc	3	Perf (20-40% fewer spills)	XL	#4
14	LICM with ownership	3	Perf (loops)	M	#1, #12
15	Escape analysis + stack alloc	3	Perf (alloc overhead)	L	None
16	Tail call optimization	3	Perf (recursion)	M	#1
17	Speculative devirtualization	3	Perf (JIT)	L	JIT system
18	Direct machine code emission	4	Compile time (15-30%)	XL	None
19	Parallel compilation	4	Compile time (4-8x)	M	None
20	Incremental compilation	4	Compile time	L	None
21	Masked auto-vectorization	4	Perf (conditional loops)	XL	#9, #5
22	Type specialization (JIT)	4	Perf (JIT)	XL	JIT system
23	DWARF debug info	4	Usability	L	None
24	Ownership-driven prefetching	4	Perf (mem latency)	L	#12

8. Specific Code Recommendations

8.1 Arena Allocation for IR Nodes

The `IrGraph` stores `data_nodes` as `Vec`. Every `clone()` of an `IrGraph` (which happens frequently during optimization, inlining, and autotuning) deep-copies all nodes. Replacing `Vec` with an arena allocator (e.g., `bumpalo` or `typed-arena`) would: (a) Eliminate all node clones (nodes are referenced by ID, the arena holds them permanently). (b) Improve cache locality (arena allocations are contiguous). (c) Reduce memory fragmentation. (d) Enable $O(1)$ deallocation of the entire graph. The `NodeId` type already serves as an index, so this change is structurally compatible. The main refactoring effort is replacing `graph.data_nodes[idx].clone()` with `graph.data_nodes[idx]` references throughout the optimizer.

8.2 IrBuilder Control Flow API

The `IrBuilder` currently lacks explicit control flow construction methods. Adding `if_else()`, `while_loop()`, and `for_loop()` builder methods that create the appropriate control nodes (`If`, `Jump`, `Region`) and `Phi` nodes would make frontend integration dramatically easier. The `axiom-ssa` crate has a `branch()` method, but it is not connected to the `IrBuilder`. These should be unified into a single, ergonomic API.

8.3 Generic Target Backend Framework

Each target backend (`x86`, `aarch64`, `riscv`, `wasm`) duplicates significant boilerplate. A generic framework that provides default implementations for common patterns (2-op vs 3-op arithmetic, condition code mapping, calling convention handling) would reduce target implementation effort by 40-60%. The `Target` trait is already well-designed; the gap is in shared `ISel` patterns and legalization rules that apply across multiple targets.

8.4 Error Recovery in Lowering

The MIR lowering currently uses `unwrap_or_else(|| VReg::new(0))` for missing node references (e.g., `crates/axiom-mir/src/lower.rs` line 157). If a `NodeId` is not found in the `node_map`, it silently substitutes `VReg(0)`, which produces incorrect code without any error indication. This should be replaced with proper error handling that either reports a compilation error or inserts an unreachable instruction. Silent substitution of `VReg(0)` is a debugging nightmare in production use.

9. Benchmarking Strategy

No optimization work is meaningful without measurement. Axiom needs a structured benchmarking framework to validate that each change improves (or at least doesn't regress) performance. The recommended approach uses three tiers of benchmarks:

9.1 Micro-Benchmarks (Compile-Time)

Measure compile-time performance for each optimization pass individually and for the full pipeline. Track: (a) Time per optimization pass (constant_fold, CSE, DSE, etc.). (b) Time per compilation phase (ownership analysis, optimization, lowering, ISel, regalloc, emission). (c) Memory usage during compilation. (d) Number of nodes before/after each pass. Use criterion.rs for statistical rigor. These should run on every PR as CI gates.

9.2 Kernel Benchmarks (Code Quality)

Measure generated code quality against LLVM -O2 and LLVM -O3 on standard computational kernels: (a) Matrix multiplication (memory-heavy, ownership-parallelizable). (b) Dot product (reduction, vectorizable). (c) Linked list traversal (pointer-heavy, ownership-chain optimization). (d) Binary search (branch-heavy, block layout sensitive). (e) Hash table lookup (random access, prefetching opportunity). (f) Sorting (comparison-heavy, branch prediction sensitive). Measure execution time, instruction count, cache miss rate, and branch misprediction rate using perf stat.

9.3 Real-World Workloads (End-to-End)

Once Axiom can compile real programs, benchmark against LLVM on: (a) SQLite query execution (database, memory-heavy). (b) regex benchmark suite (pattern matching, branch-heavy). (c) Fibonacci (recursion, tail call optimization). (d) Image processing kernels (SIMD-vectorizable). (e) JSON parsing (branch-heavy, data-dependent). These benchmarks should track Axiom vs LLVM performance ratio over time as optimizations are added, providing a clear measure of convergence toward (and eventually beyond) LLVM performance.

10. Conclusion

Axiom's architectural vision is genuinely innovative. The OwnershipRoot system represents a paradigm shift in compiler design: by making no-aliasing a structural property rather than an analysis result, it eliminates the single biggest optimization barrier in modern compilers. The sea-of-nodes IR, autotuner, tiered JIT compilation, and ThinLTO are all well-conceived and correctly architected. The codebase is clean, well-documented, and has good test coverage for implemented functionality.

However, the current implementation has critical gaps that prevent it from compiling real programs. The most urgent items are: (1) multi-block MIR lowering (without this, control flow doesn't work), (2) completing the vector ISel (without this, the flagship SIMD story is fictional), (3) implementing VRegScanner (without this, register allocation is catastrophically wasteful), and (4) adding missing IR operations including floating-point support. These four items, if addressed, would make Axiom a functional compiler backend capable of producing correct, if not yet optimal, machine code.

Beyond the critical path, the ownership system creates a unique performance ceiling that no other compiler can reach. Ownership-aware instruction scheduling, graph coloring register allocation, LICM, and prefetching are all optimizations that are dramatically simpler and more effective in Axiom than in LLVM because the no-aliasing guarantee eliminates the need for conservative assumptions. If these optimizations are implemented well, Axiom has the potential to deliver 2-5x speedup over LLVM on memory-heavy workloads, which would make it the fastest compiler backend in existence for those workloads.

The path from the current state to legendary status is long but well-defined. Each phase builds on the previous one, and each item has clear acceptance criteria. The most important thing is to start with the critical path items, establish a benchmarking framework early, and iterate rapidly. The Axiom project has the architectural foundation to become historically significant. This blueprint provides the map to get there.